

Архитектурное описание системы

1. Общие сведения о системе

Разрабатываемая система представляет собой однопользовательскую игру в жанре roguelike с консольным интерфейсом. Игрок управляет персонажем с клавиатуры, перемещается по карте уровня, взаимодействует с мобами и предметами. Предметы могут находиться на карте, подбираться в инвентарь, а также надеваться и сниматься, надетые предметы влияют на характеристики персонажа. Игра состоит из последовательности уровней, уровни могут быть заранее определёнными (загружаться из набора уровней) или генерироваться процедурно. Игра завершается победой при прохождении всех уровней либо поражением при смерти персонажа.

2. Architectural drivers

При проектировании системы учитываются следующие факторы. Архитектура должна быть расширяемой, так как дальнейшие задания предполагают развитие механик и усложнение логики. Требуется высокая скорость разработки, компоненты должны быть слабо связаны, чтобы изменения в генерации уровня, данных или рендере не приводили к переработке всего приложения. Интерфейс должен работать без заметных задержек при вводе и перерисовке. Дополнительным ограничением является простота реализации и понятность структуры для учебного проекта.

3. Роли и случаи использования

В системе присутствует один основной актор - игрок. Игрок запускает приложение и взаимодействует с игрой через клавиатуру. Система отображает состояние уровня и результаты действий пользователя.

3.1. Прецедент П1. Игра в Roguelike игру

Основной исполнитель: игрок.

Заинтересованные лица и их требования: игрок хочет получить удовольствие от игры и проникнуться духом старых игр.

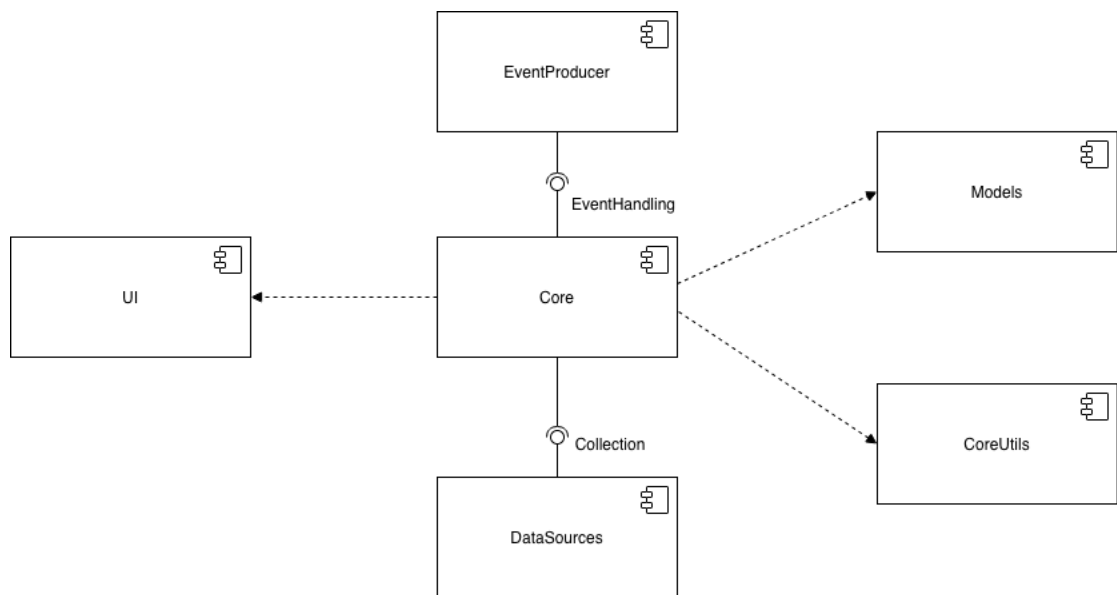
Основной успешный сценарий.

1. Игрок запускает игру.
 2. Система предлагает пользователю ввести имя персонажа, после чего пользователь вводит имя.
 3. Далее система предлагает пользователю выбрать пол, расу и класс персонажа - пользователь делает выбор.
 4. После завершения ввода параметров персонажа система генерирует новый уровень.
 5. Пользователь проходит уровень, уничтожая всех мобов.
 6. После выполнения условия завершения уровня система предлагает перейти на новый уровень.
 7. Пользователь подтверждает переход.
- Действия, описанные в пп. 5-7, повторяются, пока пользователь не пройдет все уровни.
8. После прохождения последнего уровня система поздравляет пользователя с прохождением игры.

Расширения: при закрытии пользователем терминала с игрой приложение завершается без выполнения дополнительных действий.

4. Диаграмма компонентов и описание компонентов

Система разделена на логические компоненты: EventProducer, Core, UI, Models, CoreUtils и DataSources. Поток управления в типичном цикле работы следующий: EventProducer формирует событие (Event), GameController обрабатывает событие и изменяет GameState, после чего Renderer перерисовывает состояние.

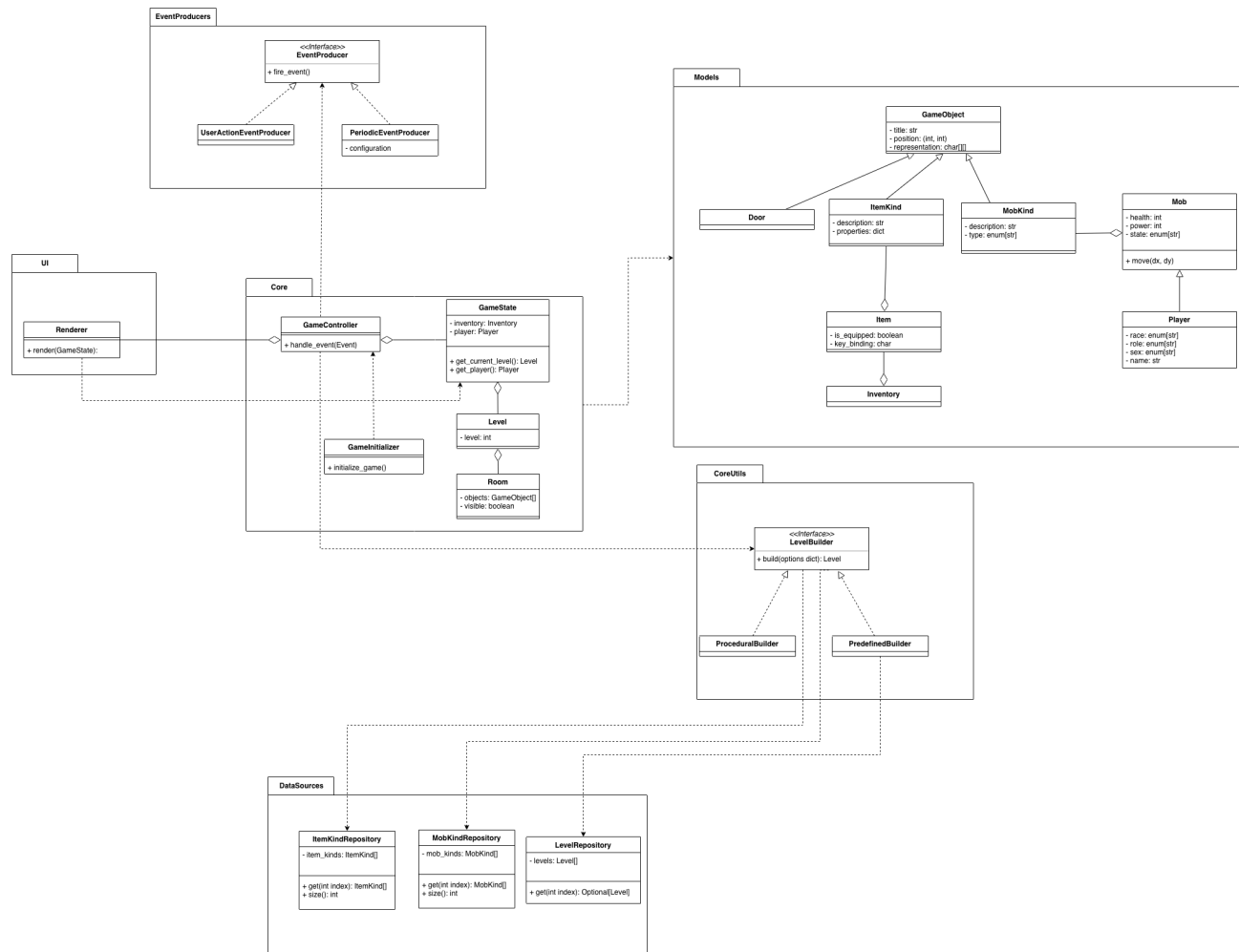


Компонент EventProducer включает интерфейс EventProducer (метод `fire_event()`) и реализации `UserActionEventProducer` и `PeriodicEventProducer` (с конфигурацией).

Компонент Core содержит `GameController` (метод `handle_event(Event)`), `GameState` (поля `inventory: Inventory` и `player: Player`, методы `get_current_level(): Level` и `get_player(): Player`), а также `GameInitializer` (метод `initialize_game()`).

Компонент UI содержит `Renderer` с методом `render(GameState)`. Компонент Models включает доменные классы `GameObject`, `Door`, `ItemKind`, `Item`, `Inventory`, `MobKind`, `Mob` и `Player`. Компонент CoreUtils содержит интерфейс `LevelBuilder` (метод `build(options: dict): Level`) и реализации `ProceduralBuilder` и `PredefinedBuilder`. Компонент DataSources содержит репозитории `ItemKindRepository`, `MobKindRepository` и `LevelRepository`.

5. Диаграмма классов и логическая структура



Ключевым классом подсистемы Core является GameController, который обрабатывает события через метод `handle_event(Event)`. Класс GameState хранит ссылки на Inventory и Player, а также предоставляет доступ к текущему уровню через `get_current_level(): Level` и к игроку через `get_player(): Player`. Модель уровня представлена классами Level (поле `level: int`) и Room (поле `objects: GameObject[]`). Инициализация выполняется через GameInitializer (метод `initialize_game()`).

В доменной модели базовым классом игровых объектов является GameObject, содержащий `title: str`, `position: (int, int)` и `representation: char[][]`. От GameObject наследуется Door. Классы ItemKind и MobKind содержат описание

(description: str) и дополнительные параметры (properties: dict у ItemKind, type: enum[str] у MobKind).

Класс Item содержит признаки is_equipped: boolean и key_binding: char. Инвентарь представлен классом Inventory и используется для хранения предметов, находящихся у игрока, а также для операций над ними (например, добавление предмета и экипировка).

Класс Mob содержит health: int, power: int, state: enum[str] и метод move(dx, dy). Класс Player является специализацией Mob и дополнительно содержит race: enum[str], role: enum[str], sex: enum[str], name: str.

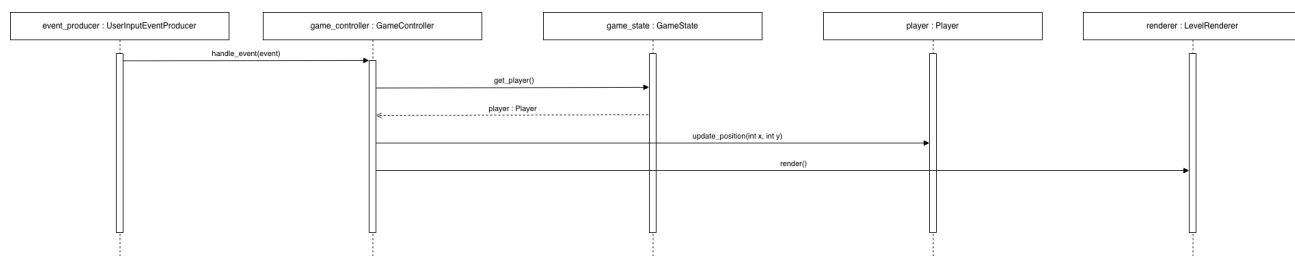
Репозитории DataSourcees реализованы классами ItemKindRepository (item_kinds: ItemKind[]), MobKindRepository (mob_kinds: MobKind[]) и LevelRepository (levels: Level[]). Для доступа используются методы size(): int и get(int index) (для LevelRepository - get(int index): Optional[Level]).

Для построения уровней применяется интерфейс LevelBuilder с методом build(options: dict): Level. Реализация ProceduralBuilder формирует уровень процедурно, а реализация PredefinedBuilder использует предопределённые уровни из LevelRepository.

6. Диаграммы последовательностей (взаимодействия)

Диаграммы последовательностей фиксируют типовые сценарии работы системы и показывают передачу управления между объектами при обработке событий пользователя.

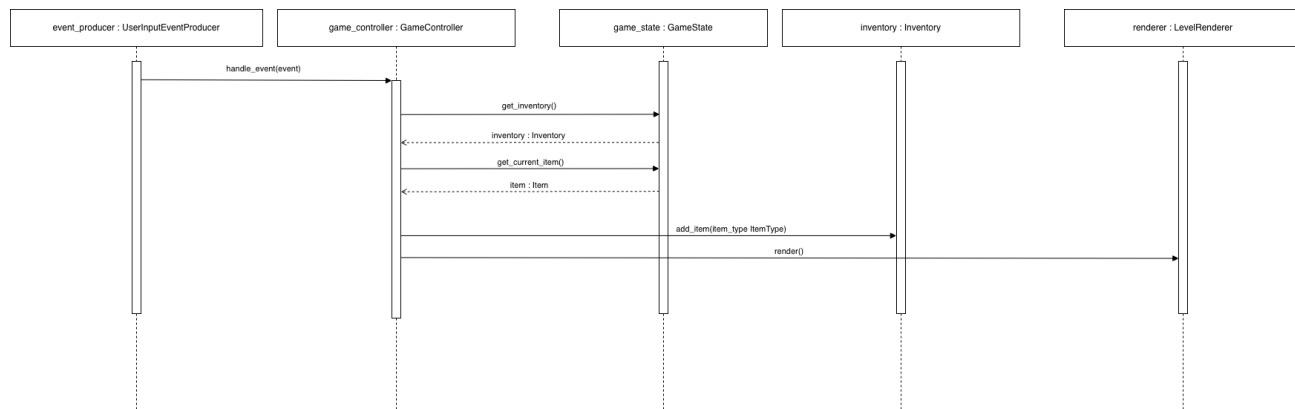
6.1. Перемещение персонажа



UserActionEventProducer формирует событие и инициирует его обработку в

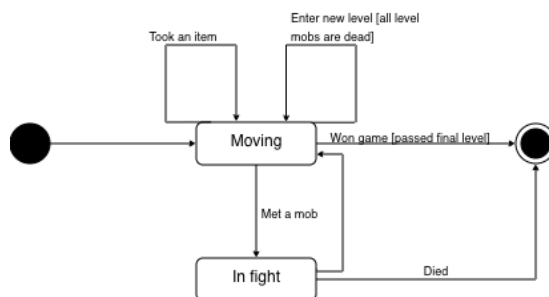
GameController.handle_event(Event). GameController получает игрока через GameState.get_player(): Player и изменяет позицию игрока (например, через метод Player.update_position(int x, int y) или эквивалентный вызов). После изменения состояния GameController вызывает Renderer.render(GameState).

6.2. Подбор предмета



UserActionEventProducer формирует событие подбора предмета и передаёт его в GameController.handle_event(Event). GameController получает доступ к Inventory (через GameState.inventory или через аксессор) и добавляет предмет в инвентарь (например, вызовом Inventory.add_item(...)). Далее выполняется Renderer.render(GameState).

7. Диаграмма состояний



Состояния игры представлены режимами Moving (исследование уровня), In fight (бой) и Game over (завершение игры). Переход Moving > In fight происходит при встрече с мобом. После боя возможен переход в Moving при победе или переход в Game over при смерти игрока. Переход на следующий уровень выполняется при выполнении условия завершения уровня и не меняет основной режим (остается

Moving), но обновляет текущий Level. При прохождении последнего уровня выполняется переход в Game over с результатом “победа”.